

Programming Language Principles, Design, and Implementation

Compilers - Garbage Collection

Vincent Rahli

University of Birmingham

Where are we?

- ▶ Part 1: Principles of programming
- ▶ **Part 2: Compilers**

Today

Garbage Collection

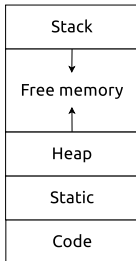
- ▶ Reference counting
- ▶ Mark-and-Sweep

Further reading:

- ▶ Chapter 7 of the Dragon book

Recall: Storage Organization

The virtual memory is typically divided into the following **program** and **data** areas:



where

- ▶ the generated target code is computed at compile time and stored in **Code**
- ▶ some objects (e.g., global constants) are also fixed at compile time and stored in **Static**
- ▶ the dynamic **Stack** stores **activation records** generated during procedure calls, and starts from a higher address and grows downward
- ▶ the dynamic **Heap** stores dynamically allocated data (e.g., using `malloc` in C), and starts from a lower address and grows upward

Problem

Efficiently manage heap-allocated memory

Two solutions

- ▶ Manual: leave it to the programmer (e.g., C & C++)
 - ▶ precise control of the memory
 - ▶ need to manage the memory & error prone, e.g., memory leaks, double frees, etc.
- ▶ Automatic: leave it to the runtime environment (e.g., Java & Haskell)
 - ▶ programmers don't have to reclaim unused resources
 - ▶ programmers cannot reclaim unused resources

Garbage: some data that cannot be referenced anymore (is unreachable)

Garbage collector (GC): reclaim the memory used by unreachable data (undecidable in general whether data is garbage)

Lisp was the first language to feature a GC: dates back to 1958

Assumptions

We will assume that:

- ▶ objects have a type indicating how much memory they take up
- ▶ references to objects point to their beginning (cannot create a reference to the middle of an object as in C)

Type-safe languages that cannot manipulate the memory arbitrarily are therefore good candidates for automatic GC.

In unsafe languages all memory locations can be referenced at all times.

Performance Metrics

Different metrics can be used:

- ▶ **Overall execution time**
- ▶ **Space usage** (compacting vs. non-compacting)
- ▶ **Pause time** (incremental vs. stop-the-world)

Tradeoffs must be made carefully:

- ▶ **trace-based collectors**: infrequent collection increases memory usage and length of the pauses
- ▶ **reference counting collectors**: no long pauses and memory efficient but constant overhead

Reachability

The set of reachable objects changes as the program executes:

- ▶ **Object allocation** introduces new reachable objects
- ▶ **Parameter passing and return values** propagate reachability
- ▶ **Reference assignments** propagate but can also terminate reachability (if the variable assigned to was the only reference to an object)
- ▶ **Procedure returns** can terminate reachability (as a frame is popped off the stack)

Termination of reachability can cause more objects to be unreachable

How can we find reachable objects?

2 basic ways to find unreachable objects:

- ▶ keep track of objects as they become unreachable (e.g., reference counting)
- ▶ periodically locate all the reachable objects (e.g., trace-based collectors)

Reference Counting

Every object stores its reference count: number of references to the object.

- ▶ object allocation: reference count is set to 1
- ▶ parameter passing: reference count of each object passed into a procedure is incremented
- ▶ reference assignment $u = v$: reference count of the object referred to by v is incremented; reference count of the old object referred to by u is decremented.
- ▶ procedure return: all the references held by the local variables are decremented.

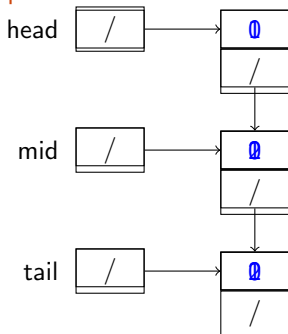
When the reference count of an object reaches 0: decrement the reference count of each object pointed to by a reference in the object

- ▶ Advantages: incremental; simple
- ▶ Disadvantages: expensive; cannot deal with cyclic data structures

Reference Counting – Example

Execute the algorithm on the following example:

```
1: class LinkedList {
2:     LinkedList next;
3: }
4:
5: int main() {
6:     LinkedList head = new
        LinkedList; LinkedList head = new
        LinkedList;
7:     LinkedList mid = new
        LinkedList; LinkedList mid = new
        LinkedList;
8:     LinkedList tail = new
        LinkedList; LinkedList tail = new
        LinkedList;
9:     head.next = mid; head.next = mid;
10:    mid.next = tail; mid.next = tail;
11:    mid = tail = null; mid = tail = null;
12:    head.next.next =
        null; head.next.next = null;
13:    head = null; head = null;
14: }
```

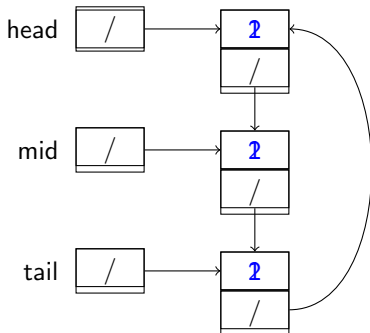


Reference Counting – Example

Cannot deal with cyclic data structures!

What happens with this example?

```
1: class LinkedList {  
2:     LinkedList next;  
3: }  
4:  
5: int main() {  
6:     LinkedList head = new  
    LinkedList; LinkedList head = new  
    LinkedList;  
7:     LinkedList mid = new  
    LinkedList; LinkedList mid = new  
    LinkedList;  
8:     LinkedList tail = new  
    LinkedList; LinkedList tail = new  
    LinkedList;  
9:     head.next = mid; head.next = mid;  
10:    mid.next = tail; mid.next = tail;  
11:    tail.next = head; tail.next = head;  
12:    head = mid = tail = null; head =  
    mid = tail = null;
```



Trace-Based Collection

Instead of collecting garbage as it is created:

- ▶ find unreachable objects periodically
- ▶ e.g., when the free space is below some threshold

Mark-and-Sweep

Intuition:

- ▶ stop the algorithm
- ▶ visit and **mark** all **reachable** objects
- ▶ **sweep** the heap to free up **unreachable** objects

Starts from a known set of reachable objects: **the root set**.

The root set: the objects that can be accessed directly by a program, without having to dereference pointers (immediately known to be reachable).

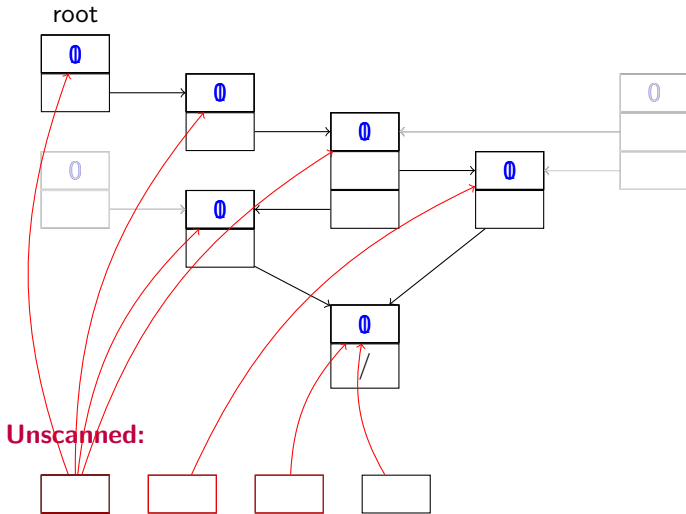
Algorithm at a high level:

- ▶ **input:** a root set of objects, a heap, a **Free** list with unallocated chunks of the heap
- ▶ **output:** a modified **Free** list
- ▶ **method:**
 - ▶ keep track in a **Unscanned** list of the reachable objects, whose successors have not been checked yet.
 - ▶ once an object is reached, mark it using a **reached-bit**

Mark-and-Sweep – Algorithm

```
1: // marking
2: for  $o \in$  the root set
3:   set the reached-bit of  $o$  to 1
4:   add  $o$  to Unscanned
5:
6: while Unscanned  $\neq \emptyset$ 
7:   remove some object  $o$  from Unscanned
8:   for  $o' \in$  the objects referenced in  $o$ 
9:     if the reached-bit of  $o'$  is 0 (i.e., unreachable) then
10:       set the reached-bit of  $o'$  to 1
11:       add  $o'$  to Unscanned
12:     end if
13:
14:
15: // sweeping
16: for  $o \in$  the heap
17:   if the reached-bit of  $o$  is 0 (i.e., unreachable) then
18:     add  $o$  to Free
19:   else
20:     set the reached-bit of  $o$  to 0
21:   end if
22:
```

Mark-and-Sweep – Example



What will happen? Reclaiming the memory for unmarked (unreachable) objects, and resetting reached-bits

Mark-and-Sweep

Sweeping is expensive: needs to go through the entire heap

Baker's algorithm fixes this by maintaining a list of allocated objects

4 lists:

- ▶ **Free**: does not hold a reachable object
- ▶ **Unreached**: presumed unreachable until proved reachable (allocated objects)
- ▶ **Unscanned**: reachable but its pointers haven't been checked yet
- ▶ **Scanned**: reachable and scanned

2 bits are used to know what state each object is in.

Mark-and-Sweep – Baker's Algorithm

```
1: Scanned =  $\emptyset$ 
2: Unscanned = root set
3: remove root set from Unreached
4: while Unscanned  $\neq \emptyset$ 
5:     move  $o$  from Unscanned to Scanned
6:     for  $o' \in$  the objects referenced in  $o$ 
7:         if  $o' \in$  Unreached (i.e., yet to be scanned) then
8:             move  $o'$  from Unreached to Unscanned
9:         end if
10:
11:
12: // Unreached contains the known unreachable objects
13: Free = Free  $\cup$  Unreached
14: // Scanned contains the updated list of allocated objects
15: Unreached = Scanned
```


Beyond Mark-and-Sweep

Mark-and-Compact: relocates objects into one part of the heap to avoid memory fragmentation (needs to find free space).

Cheney's copying collector: partitions the memory into 2 regions, uses a single partition, and switches partition when GC (to avoid having to find free space).

Running time of the algorithms:

- ▶ **basic Mark-and-Sweep:** proportional to the number of chunks in the heap
- ▶ **Barker's algorithm:** proportional to the number of reached objects
- ▶ **basic Mark-and-Compact:** proportional to the number of chunks in the heap + the total size of the reached objects
- ▶ **Cheney's algorithm:** proportional to the total size of the reached objects

Conclusion

What did we cover today?

- ▶ Garbage collection
- ▶ Reference counting
- ▶ Mark-and-Sweep